药材烘干过程的耦合传热传质模型与数值仿真

摘　要

干燥过程中的温度传递与水分迁移将直接影响中药材的加工品质与生产效率。针对药材热风烘干过程中**温度场、含水率场**具有显著的**时空非均匀性**，且**物性参数与药材尺寸随含水率变化而动态变化**的问题，本文以圆柱形药材为研究对象，建立一维径向耦合传热传质模型，并采用有限差分法结合隐式$Euler$时间离散进行数值求解，逐步考虑横物性、变物性及尺寸收缩对烘干过程的影响。

针对问题一：建立恒物性条件下的**热质传递模型**，并对药材预热阶段进行模拟。通过计算得到烘干 1800s 时，药材中心温度约为 33.5313℃，表面温度约为 36.7251℃，中心干水基含水率约为 2.55，表面降至 1.5447，表明了预热阶段已经形成了明显的径向温度与水分梯度。

针对问题二：进一步考虑药材物性参数随含水率变化的实际特征，建立**变物性非线性热质传递模型**，结果表明，药材温度在烘干初期快速升高，约 3h 后逐渐接近烘房环境温度；水分持续由内部向表面迁移，随着表层含水率降低，后期烘干逐渐转变为**内部水分扩散控制**。

针对问题三：以药材内部所有位置的干基含水率均低于$0.15\ kg/kg$作为烘干完成的判定条件，以内部最大含水率作为终点判据，计算得到固定尺寸条件下药材达到要求所需时间约为 57.8753h。

针对问题四：利用药材半径随时间变化数据建立**移动边界模型**，并通过无量纲坐标变换处理收缩边界。计算得到考虑尺寸收缩后药材达到同一含水率要求所需时间约为 53.2061h，较固定尺寸模型缩短约 4.6692h，降幅约 8.07%，说明在本题的参考条件下，尺寸收缩对后期水分迁移具有一定促进作用。

最后通过**网格收敛性**、时间步长、**边界残差**及质量守恒等检验对模型进行验证，并开展关键参数敏感性分析。结果表明，所建模型具有较好的数值稳定性和可靠性，能够较为准确的地描述药材烘干过程中温度、水分、物性参数及尺寸的动态演化，可为药材烘干时间确定与工艺优化提供数学依据。

关键词：药材烘干，耦合传热传质，变物性，非线性水分扩散，移动边界，数值模拟

一、 问题重述

1.1 问题背景

药材干燥是中药材加工过程中影响产品品质、生产效率与能源利用的重要环节。热风干燥过程中，外部热空气通过药材表面向内部传递热量，同时内部水分在浓度梯度驱动下向表面迁移并排出。因此药材烘干本质上是一个具有明显非稳态、非均匀和热质耦合特征的传热传质过程。

对于近似圆柱形的药材，其内部温度和含水率在不同径向位置存在明显的差异。随着干燥持续进行，药材含水率逐渐降低，使整个过程表现出显著的非稳态特征，药材的密度、比热容、导热系数以及有效水分扩散系数等物性参数也会随之变化。此外，持续失水还会导致药材自身尺寸收缩，使水分迁移路径、边界位置发生动态变化，因此，仅采用恒物性、固定尺寸模型难以完整描述实际烘干过程。

本题给定近似圆柱形药材，其长度为$25cm$，初始半径为$2cm$，初始温度为$28℃$，初始干基含水率为2.55。同时给出了烘房温度与环境含水率的变化数据、药材半径随时间变化的数据，以及不同阶段对应的物性参数经验公式。

针对上述条件，本文以药材内部温度场和干基含水率场为核心研究对象，在柱坐标系下建立径向热质传递模型，并依次考虑恒物性、变物性以及尺寸收缩的影响，最终确定满足规定含水率要求的合理烘干时间。

1.2 问题提出

1. 问题一：恒物性条件下预热过程的热质传递规律

在烘干初始阶段，药材温度与烘房环境之间存在较大的温差，药材内部含水率较高。假设药材几何尺寸和物性参数保持不变，根据初始条件及烘房温度、环境含水率随时间变化的数据，建立药材内部温度和干基含水率的径向传递模型。

模型需要考虑药材内部热传导、水分扩散以及药材表面与外部环境之间的对流换热和传质，并利用轴心处的对称性建立中心边界条件。通过数值求解，获得$0\sim1800s$药材内不同时间和不同径向位置处的温度和含水率分布，为后续变物性条件下的全过程模拟提供初始基础和参数衔接。

2. 问题二：变物性条件下药材全过程烘干规律

在问题一的基础上，引入题目附录给出的物性参数经验公式，使密度、比热容、导热系数以及有效水分扩散系数随局部含水率和温度动态变化。

由此建立变物性非线性热质传递模型，模拟整个烘干过程中温度场与含水率

的动态演化，并分析预热阶段向恒温干燥阶段过渡后，温度均匀化与水分梯度演化之间的关系。

3. 问题三：基于全域含水率约束的烘干终点判定

在实际生产过程中，药材是否达到烘干要求不能仅依据表面含水率或整体平均含水率进行判断，而应确保药材内部各位置的含水率均达到规定标准。根据题目要求，药材任意位置的干基含水率均应低于$0.15\,kg/kg$，因此需要基于问题二得到的时空含水率场中确定满足全域约束的最早时刻。

定义时刻t药材内部最大干基含水率为

$$C_{max}(t) = \max_{0\le r\le R} C(r,t) \tag{1}$$

将首次满足

$$C_{max}(t) < 0.15 \tag{2}$$

的时刻定义为药材的最短达标烘干时间。在此基础上，分析最大含水率的时空演化规律，识别烘干过程中最难达到含水率要求的控制区域，并确定药材从初始状态达到最终质量约束所需的时间。

4. 问题四：考虑尺寸收缩的移动边界烘干模型

在药材持续失水的过程中，其体积会随含水率降低而逐渐收缩，使药材半径由初始值随时间动态变化、实际计算区域和水分迁移距离会同步随烘干过程发生变化。因此，问题四在变物性模型基础上进一步考虑药材的几何收缩效应，建立具有移动边界特征的热质传递模型。

根据附件提供的半径—时间数据，构造连续的半径变化函数R（t），并通过坐标变换将实际变化的径向区域映射到固定计算区域，从而在统一数值框架下求解收缩条件下药材内部温度与含水率的动态变化。

最终仍以问题三的全域含水率约束作为烘干终点判据，确定考虑尺寸收缩后的烘干时间，并与固定尺寸模型的结果进行定量比较，以分析尺寸收缩对水分迁移过程及烘干效率的影响。

综上，四个问题按照模型复杂程度逐步递进，形成“恒物性→变物性→达标判定→移动边界”的完整建模体系，为后续模型求解和结果分析提供统一框架。

二、 问题分析

2.1 总体概述

药材热风烘干本质上是一个传热与传质相互耦合的非稳态过程。外部热空气通过药材表面向内部传递热量，同时内部水分在浓度作用下向表面迁移[1]。随着

烘干持续进行，药材温度和含水率也随之不断变化，并进一步引起物性参数的改变；持续失水还会造成药材尺寸收缩，使得实际传热传质区域随时间发生变化。

考虑到药材长度远大于半径，且初始状态及外部环境沿轴向基本均匀，本文忽略轴向变化，仅研究径向传热与传质，将实际过程简化为柱坐标系下的一维径向耦合模型。四个问题并非相互独立，而是围绕同一物理过程逐层增加影响因素：首先研究恒物性条件下的预热过程，再引入含水率和温度相关的变物性，随后依据全域含水率要求确定烘干点，最后进一步考虑失水收缩造成的移动边界效应，形成由基础模型向高精度模型逐步扩展的建模体系。

2.2 问题一分析：恒物性条件下的预热过程

问题一主要研究烘干初期药材内部温度和含水率的时空变化规律，其关键在于准确描述外部环境与药材内部之间的双重传递过程。一方面，药材表面与热空气之间存在温差，热量由表面向内部逐渐传递；另一方面，由于药材内部含水率较高，在浓度梯度驱动下，水分由内部持续向表面迁移。因此，烘干过程中药材内部的温度场和水分场具有明显的径向非均匀性，不能采用整体平均温度或平均含水率进行简化描述，需要建立基于空间分布的热质传递模型，对其内部温度和水分的动态变化进行刻画。

本问假定药材尺寸及物性参数保持不变，在此基础上建立径向传热与传质模型，并分别采用轴心对称条件和表面对流交换条件刻画药材中心及边界处的物理过程。对于环境温度和环境含水率随时间变化的问题，将直接利用附件数据进行插值，使边界驱动力能够连续随时间更新，而非采用单一平均环境参数。该处理既保留了题目实验数据中的动态信息，又为后续变物性全过程模型提供可靠的初始场与数值求解框架。

2.3 问题二分析：变物性条件下的全过程烘干

问题二相比问题一的主要难点在于药材并非具有恒定物性。随着水分不断逸出，其密度、比热容、导热系数和有效水分扩散系数均会发生变化，而这些参数又反过来影响温度和水分的进一步迁移[2]。因此，模型由线性传递问题转变为状态变量与物性参数相互反馈的非线性耦合问题。

为避免采用平均物性参数造成较大的模型偏差，本文直接将题目附录给出的经验关系嵌入控制方程，使不同空间位置的物性参数根据当地温度和含水率实时更新。数值求解时采用隐式时间推进与迭代修正相结合的方法，在每一个时间层内协调温度、含水率及物性参数之间的一致性，从而提高长时间模拟的稳定性和可靠性。同时，将烘房环境数据完整纳入边界条件，使模型能够同时反映环境变

化、内部扩散和物性演化三方面影响。该模型也是后续确定烘干终点及研究收缩效应的核心基础。

2.4 问题三分析：全域约束下的烘干终点

问题三的关键并不是计算某一位置的含水率，而是确定药材整体达到规定干燥程度的最早时刻。题目要求药材各处干基含水率均低于 0.15，因此采用平均含水率或者表面含水率作为判断依据，都可能出现“局部达标、内部未达标”的情况，无法严格满足题目的质量要求。

针对这一特点，本文将药材内部最大的干基含水率作为烘干终点的控制指标。由于烘干过程中表层首先失水，而内部水分迁移存在一定的滞后性，最大含水率通常能够较好地表征药材内部最难干燥区域的含水状态。因此，以最大含水率首次低于规定阈值作为终点判据，本质上是将题目的“各处均达标”转化为一个明确的全域约束问题。进一步结合问题二获得的完整时空场数据，对终点进行时间定位，而不是简单地选取离散计算时刻。该处理能够避免过早结束导致的药材内部残余水分超标，同时减少不必要的延长烘干，为确定满足质量要求下的最短烘干时间提供依据。

2.5 问题四分析：考虑尺寸收缩的移动边界

问题四进一步考察药材失水后的几何收缩效应。附件数据表明，随着烘干时间的增加，药材半径将持续减小，此时传热传质区域不再固定[3]。若仍采用问题二中的固定半径假设，虽然能够描述物性变化，但无法反映扩散路径缩短以及边界位置变化对内部水分迁移的影响[4]。因此，本问的核心是将材料状态变化与几何边界变化统一起来。

本文利用附件 2 中的半径—时间数据构造连续的$R(t)$，并通过无量纲坐标变换将随时间变化的实际区域映射至固定计算区间，从而避免每一时刻重新划分网格所带来的数值困难。在此基础上，将边界运动产生的附加输运效应纳入控制模型，并与问题二的变物性模型耦合求解。该方法能够在保持统一空间网格的同时准确追踪药材实际边界，实现物性演化—水分迁移—尺寸收缩的协同描述。最后沿用问题三的全域达标标准，与固定尺寸结果进行对照，从而定量评价收缩对烘干时间和传质效率的实际影响。

三、 模型假设

为了简化问题，便于模型的建立和求解，我们现做出如下合理假设。

1. 假设药材为均质连续介质，其物性参数仅由局部含水率和温度确定。
2. 假设药材轴向变化可以忽略，只考虑径向的传热和传质。
3. 假设药材内部不存在独立的体积热源，热量仅仅由外界热空气通过表面传入。
4. 假设药材内部水分迁移可以等效为以浓度梯度作为驱动力的有效扩散过程。
5. 假设药材表面与烘房之间传热传质均采用第三类对流边界交换。
6. 假设恒温干燥阶段环境温度与水分浓度保持恒定。
7. 假设药材干燥时发生轴对称径向收缩，无轴向变形。

四、 符号说明

为保证全文数学符号定义一致，并且能够便于后续模型建立、数值离散及结果分析，本文主要符号及其含义如下表所示。

表 1 本文主要符号说明

符号	符号说明	单位
$\mathbf{r}$	径向坐标	m
$\mathbf{R}$	药材当前半径	m
$\mathbf{t}$	烘干时间	s
$\mathbf{T}$	药材内部温度	℃
$\mathbf{T_a}$	烘房环境温度	℃
$\mathbf{C}$	干基水分浓度	kg/kg
$\mathbf{C_a}$	环境水分浓度	kg/kg
$\boldsymbol{\rho}$	药材密度	kg/m^3
$\mathbf{c_p}$	比热容	$J/（kg \cdot K）$
$\mathbf{k}$	导热系数	$W/（m \cdot K）$
$\mathbf{D}$	有效水分扩散系数	m^2/s
$\mathbf{h}$	对流换热系数	$W/（m^2 \cdot K）$
$\mathbf{h_m}$	表面对流传质系数	m/s
$\Delta\mathbf{r}$	径向空间步长	m
$\Delta\mathbf{t}$	时间步长	s
$\boldsymbol{\xi}$	归一化径向坐标	—
$\mathbf{R(t)}$	随时间变化的药材半径	m
$\boldsymbol{\alpha}$	热扩散系数	m^2/s

五、 模型建立与求解

药材热风烘干过程中涉及热量传递、水分迁移、物性变化以及药材尺寸收缩等多个因素。为逐步提高模型对实际烘干过程的描述能力，本文将按照题目四个问题的递进关系建立数学模型。

问题一在药材尺寸和主要热物性保持不变的条件下，建立径向耦合传热传质模型，研究烘干初期药材内部温度场和含水率场的变化；问题二进一步引入题目给出的变物性关系，建立非线性耦合模型并模拟全过程烘干；问题三在问题二的计算结果基础上，将药材内部全域含水率约束作为判据确定最短烘干时间；问题四进一步考虑药材失水造成的半径收缩，通过移动边界模型分析尺寸变化对烘干过程的影响。

数值求解过程中，各问题采用统一的径向离散思想，并结合隐式时间推进方法进行计算。对于非线性物性参数，通过迭代更新实现温度、含水率与物性参数之间的自洽耦合。

5.1 问题一：恒物性条件下的耦合传热传质模型

5.1.1 温度场控制方程

考虑药材长度远大于半径，且烘房环境和药材初始状态沿轴向基本均匀，因此忽略轴向传热，仅考虑径向传热。在圆柱坐标系下，根据能量守恒定律，药材内部温度场满足

$$\rho c_p \frac{\partial T}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left(kr\frac{\partial T}{\partial r}\right) \tag{3}$$

问题 1 中ρ、c_p和k均为常数，因此定义热扩散系数：

$$\alpha = \frac{k}{\rho c_p} \tag{4}$$

则温度控制方程可写为：

$$\frac{\partial T}{\partial t} = \alpha\frac{1}{r}\frac{\partial}{\partial r}\left(r\frac{\partial T}{\partial r}\right) \tag{5}$$

根据题目给定参数，取

$$\rho = \frac{820kg}{m^3} \tag{6}$$

$$\mathrm{c_p} = \frac{2600\mathrm{J}}{\mathrm{kg}\cdot\mathrm{K}} \tag{7}$$

$$\mathrm{k} = \frac{0.36\mathrm{W}}{\mathrm{m}\cdot\mathrm{K}} \tag{8}$$

因此有

$$\alpha = \frac{0.36}{820\times 2600} \approx 1.689\times\frac{10^{-7}m^2}{s} \tag{9}$$

该方程描述了热空气提供的热量由药材表面逐渐向内部扩散的过程[5]。

5.1.2 水分控制方程

药材内部水分迁移主要表现为由高含水区域向低含水区域的扩散过程。根据有效扩散假设，在柱坐标系下建立水分传递方程[6]：

$$\frac{\partial C}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left(rD\frac{\partial C}{\partial r}\right) \tag{10}$$

根据题目附录 2，有效水分扩散系数为：

$$D(C) = 7\times 10^{-9}\exp\left(-\frac{0.89}{C}\right) \tag{11}$$

因此，虽然问题一中的热物性参数保持不变，但有效水分扩散系数仍随局部含水率变化，因此水分控制方程本身具有一定的非线性。

5.1.3 初始条件与边界条件

药材初始温度和干基含水率分别为：

$$\mathrm{T}(r,0) = 28, \qquad 0\le r\le R_0 \tag{12}$$

$$C(r,0) = 2.55, \qquad 0\le r\le R_0 \tag{13}$$

其中：

$$R_0 = 2cm = 0.02m \tag{14}$$

由于药材中心关于轴线对称，中心不存在净热流和水分通量，因此有：

$$\frac{\partial T}{\partial r}|_{r=0} = 0 \tag{15}$$

$$\frac{\partial C}{\partial r}|_{r=0} = 0 \tag{16}$$

药材表面与烘房空气之间采用第三类对流边界条件。温度边界为：

$$-k\frac{\partial T}{\partial r}|_{r=R} = h(T_R - T_a) \tag{17}$$

其中：

$$h = 25\,W/(m^2 \cdot K) \tag{18}$$

水分边界为：

$$-D\frac{\partial C}{\partial r}|_{r=R} = h_m(C_R - C_a) \tag{19}$$

其中：

$$h_m = 8 \times 10^{-7}\,m/s \tag{20}$$

其中$\mathrm{T_a}$（t）和$\mathrm{C_a}$（t）分别为烘房环境温度和环境水分浓度，由附件 1 数据插值得到。超过附件数据时间范围后，保持末时刻环境参数不变。

5.1.4 空间离散与中心节点处理

沿药材半径方向划分$N_r = 41$个节点：

$$r_j = j\Delta r, \qquad j = 0,1,\dots,N_r - 1 \tag{21}$$

其中

$$\Delta r = \frac{R_0}{N_r - 1} = 0.05cm = 5 \times 10^4 m \tag{22}$$

对于内部节点，采用守恒形式进行有限体积离散。以温度方程为例：

$$(\rho c_p)_j \frac{T_j^{n+1} - T_j^n}{\Delta t} = \frac{1}{r_j \Delta r}\left[r_{j+\frac{1}{2}} k_{j+\frac{1}{2}} \frac{T_{j+1}^{n+1} - T_j^{n+1}}{\Delta r} - r_{j-\frac{1}{2}} k_{j-\frac{1}{2}} \frac{T_j^{n+1} - T_{j-1}^{n+1}}{\Delta r}\right] \tag{23}$$

水分方程采用相同的守恒离散思想：

$$\frac{C_j^{n+1} - C_j^n}{\Delta t} = \frac{1}{r_j \Delta r}\left[r_{j+\frac{1}{2}} D_{j+\frac{1}{2}} \frac{C_{j+1}^{n+1} - C_j^{n+1}}{\Delta r} - r_{j-\frac{1}{2}} D_{j-\frac{1}{2}} \frac{C_j^{n+1} - C_{j-1}^{n+1}}{\Delta r}\right] \tag{24}$$

对于中心节点$r = 0$，利用轴对称条件以及$r \to 0$时的极限关系，温度场离散形式为：

$$(\rho c_p)_0 = \frac{T_0^{n+1} - T_0^n}{\Delta t} = \frac{4k_{1/2}}{\Delta r^2}(T_1^{n+1} - T_0^{n+1}) \tag{25}$$

水分方程同理：

$$\frac{C_0^{n+1} - C_0^n}{\Delta t} = \frac{4D_{2/1}}{\Delta r^2}(C_1^{n+1} - C_0^{n+1}) \tag{26}$$

这种处理避免直接计算$1/r$所造成的数值奇异问题，同时保持了圆柱几何下

的守恒性质。

5.1.5 时间离散与数值求解

时间方向采用隐式$Euler$格式：

$$\frac{T_j^{n+1} - T_j^n}{\Delta t} \tag{27}$$

和

$$\frac{C_j^{n+1} - C_j^n}{\Delta t} \tag{28}$$

问题一取时间步长

$$\Delta t = 60s \tag{29}$$

空间离散后形成三对角线性方程组，采用$Thomas$算法进行求解。对于水分扩散系数D（C），在每个时间步内根据当前含水率更新扩散系数，并通过迭代获得收敛解。

5.1.6 问题一计算结果

基于上述模型进行数值求解，计算时间范围取$0\sim1800s$，时间步长为$1s$，径向空间步长为$0.1cm$。按照题目要求，在典型时刻和典型径向位置提取温度及干基水分浓度结果，完整计算数据保存于 result1.xlsx。

1. 温度场结果

表 2 给出了预热阶段不同时间和径向位置处的药材温度。

表 2 30 分钟内药材的温度

时间/s	到药材中心的距离/cm				
	0	0.5	1	1.5	2
100	28.0001	28.0004	28.0040	28.0319	28.1771
300	28.0409	28.0633	28.1501	28.3634	28.8384
600	28.4494	28.5311	28.7958	29.3018	30.1414
900	29.3117	29.4443	29.8575	30.5911	31.6946
1200	30.5194	30.6851	31.1934	32.0768	33.3834
1500	31.9616	32.1513	32.7268	33.7012	35.0668
1800	33.5313	33.7268	34.3155	35.3083	36.7251

由表 2 可见，预热过程中药材温度整体持续升高，且始终呈现表面高、中心低的径向分布特征。$1800s$时中心与表面温度分别为$33.5313℃$和$36.7251℃$，温差为$3.1938℃$。说明$30min$后药材内部仍然没有完全达到均温状态，热量仍在由

表面向内部传递。

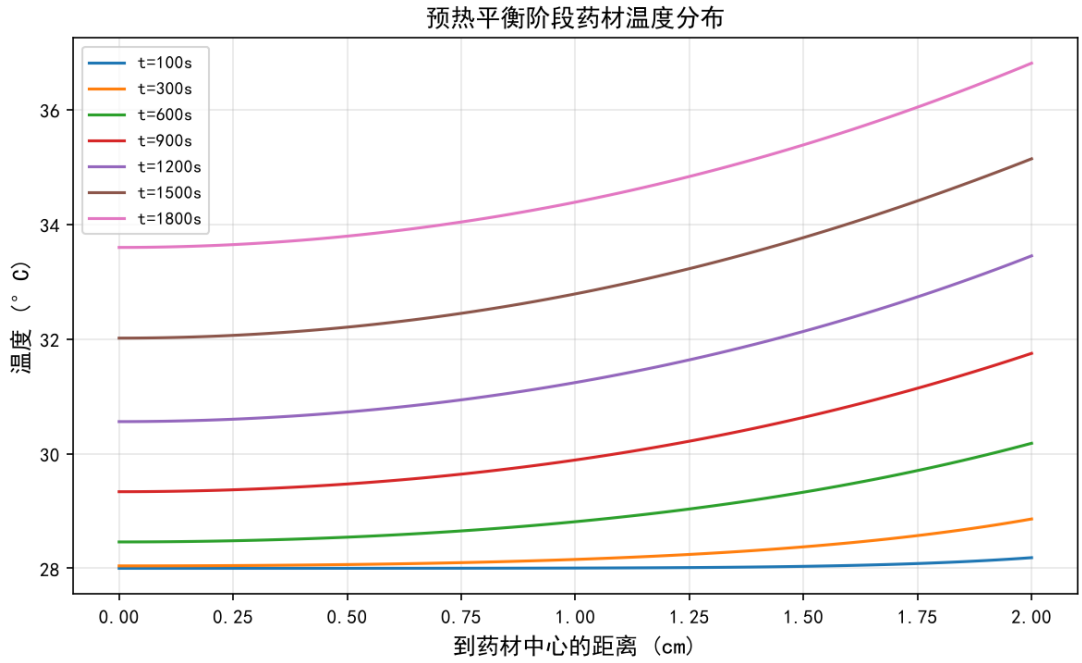


图 1 预热平衡阶段药材温度分布

图 1 进一步给出了1800s时的径向温度分布。温度曲线连续平滑，并由中心向表面逐渐升高，与热量由外向内传递的物理过程一致，也验证了中心对称边界条件下数值解的合理性。

2. 水分场结果

在温度场逐渐升高的同时，药材内部水分也发生径向迁移。表 3 给出了相同时间节点和径向位置处的干基水分浓度

表 3 30 分钟内药材的水分浓度

时间/s	到药材中心的距离/cm				
	0	0.5	1	1.5	2
100	2.5500	2.5500	2.5500	2.5500	2.2670
300	2.5500	2.5500	2.5500	2.5490	2.0727
600	2.5500	2.5500	2.5500	2.5355	1.9065
900	2.5500	2.5500	2.5496	2.5060	1.7833
1200	2.5500	2.5500	2.5481	2.4674	1.6895
1500	2.5500	2.5499	2.5446	2.4294	1.6115
1800	2.5500	2.5497	2.5385	2.3811	1.5447

由表 3 可见，预热阶段水分迁移具有明显的径向不均匀性。1800s时中心干基水分浓度仍约为2.55kg/kg，而表面降至1.5447kg/kg，表明水分主要由内部

向表面迁移并排出，内部水分响应明显滞后于表面。

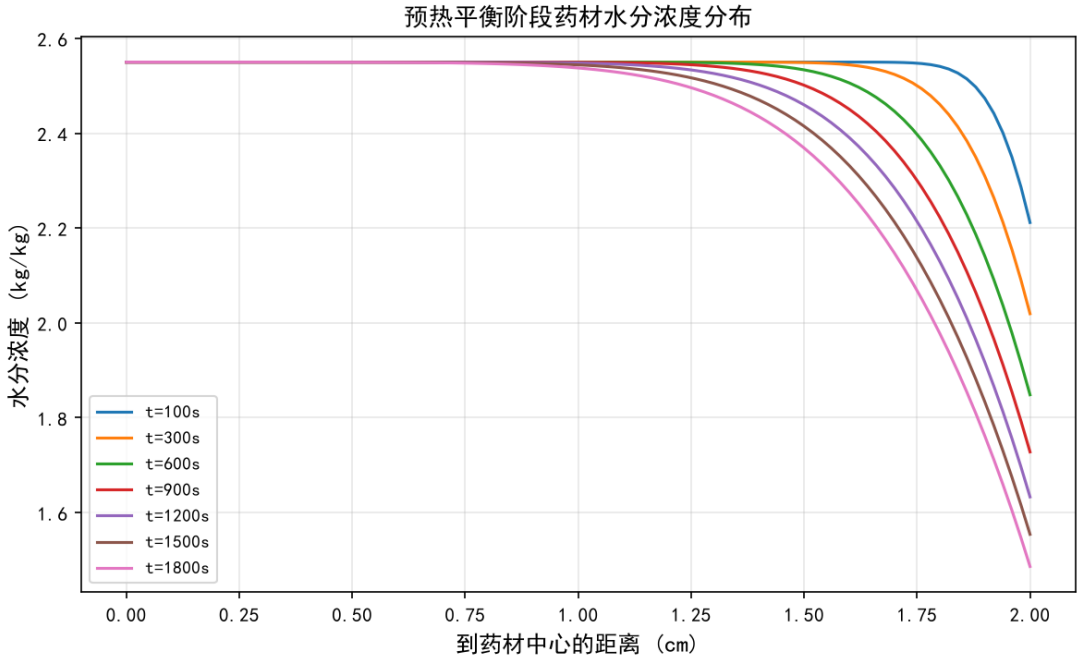


图 2 预热平衡阶段药材水分浓度分布

图 2 显示，1800s时水分浓度由中心向表面逐渐降低，形成明显的径向浓度梯度。该梯度为后续干燥阶段水分持续向表面迁移提供了主要驱动力，因此问题一得到的水分场不能简单等效为均匀分布。

3. 温度场时空演化

为进一步刻画预热过程中热量由表面向内部的传播过程，绘制0~1800s内温度场时空分布热力图，如图 3 所示。

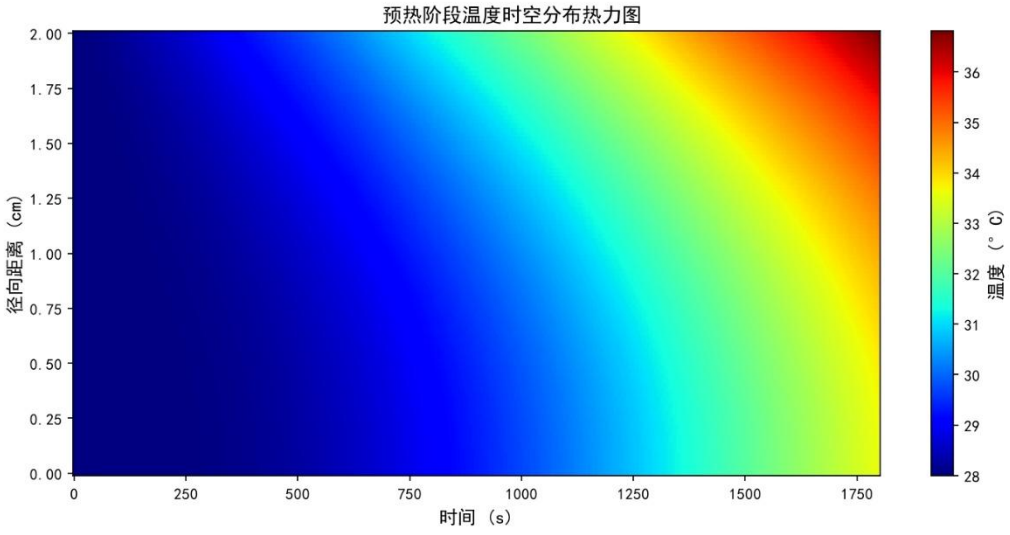


图 3 预热阶段温度时空分布热力图

由图 3 可直观看出，升温首先发生于药材表层，随后高温区域逐渐向中心扩展；随着时间增加，温度梯度逐步减小，但$1800s$时仍未完全消失。因此，预热过程具有明显的“表面快速响应—内部逐步升温—温差逐渐减小”特征。

综合表 2、表 3 及图 1～图 3 可知，预热阶段温度场和水分场具有不同的响应速度：温度由外向内快速传递，而水分迁移相对滞后，并形成显著的径向梯度。因此，将$1800s$时的T（r，1800）与C（r，1800）作为问题二的初始场，比采用统一平均温度和平均含水率更符合实际干燥过程。

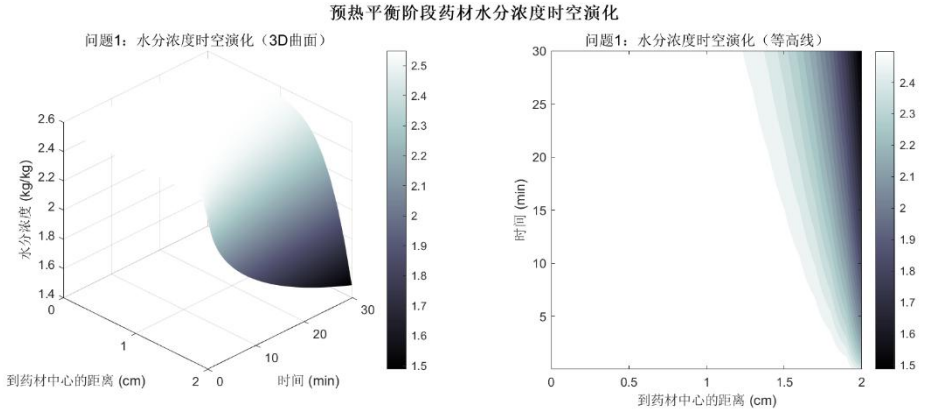


图 4 预热平衡阶段药材水分浓度时空演化

由图 4 可见，预热过程中药材表层水分浓度首先降低，并逐渐向内部扩展。相比温度场，水分变化更加缓慢，表明水分迁移存在一定的时间滞后性，为后续耦合传热传质模型的建立提供依据。

综上，问题一完成了药材预热阶段温度场和水分场的确定，并为后续变物性全过程干燥模型提供了连续、非均匀的初始状态。

5.2 问题二：变物性条件下的非线性耦合模型

问题一中将药材的密度、比热容和导热系数视为常数，能够较好描述预热阶段的基本传热传质规律。但随着烘干持续进行，药材含水率不断下降，其热物性和水分扩散特性随之发生变化[7]。为提高模型对实际烘干过程的刻画能力，本节在问题一模型基础上引入题目附录 3 给出的变物性关系，使ρ、c_p、k和D随局部含水率及温度动态更新，从而建立变物性条件下的非线性耦合模型。

5.2.1 变物性控制方程

考虑药材密度、比热容、导热系数和有效水分扩散系数均随状态变量变化，控制方程仍采用柱坐标下的一维径向传热传质形式：

$$\rho(C)_{C_P}(C)\frac{\partial T}{\partial t}=\frac{1}{r}\frac{\partial}{\partial r}\left(k(C)r\frac{\partial T}{\partial r}\right) \tag{30}$$

水分控制方程为：

$$\frac{\partial C}{\partial t}=\frac{1}{r}\frac{\partial}{\partial r}\left(rD(C,T)\frac{\partial C}{\partial r}\right) \tag{31}$$

其中，按照题目附录 3 给出的物性关系为：

$$\rho(C)=650+128C \tag{32}$$

$$c_p(C)=1450+2736\frac{C}{C+1} \tag{33}$$

$$k(C)=0.21+0.38\frac{C}{C+1} \tag{34}$$

以及

$$D(C,T)=2.4\times10^{-3}exp\left(-\frac{0.45}{C}\right)exp\left(-\frac{3850}{T+273.15}\right) \tag{35}$$

内部温度变化会影响水分扩散，而含水率变化又会反过来改变药材的导热、储热和扩散能力，形成明显的热—质双向耦合。

在数值求解过程中，采用有限体积法进行空间离散，并利用隐式$Euler$格式进行时间推进；每个时间步内通过$Picard$迭代同步更新温度场、水分场及物性参数，直至相邻两次迭代结果满足收敛条件。

问题一计算得到的$t=1800s$时温度场和水分场作为问题二的初始状态，从而保证两个模型之间的状态连续。

5.2.2 在 3 小时内药材温度场计算结果

对变物性模型进行$0\sim3h$数值模拟，选取具有代表性的时间节点和径向位置提取结果。给出了药材不同位置的温度变化。

表 4 3 小时内药材的温度

时间/h	到药材中心的距离/cm				
	0	0.5	1	1.5	2
0.5	32.1554	32.3468	32.9264	33.9145	35.3571
1.0	40.3115	40.4832	40.9896	41.8068	42.9280
1.5	45.7854	45.8741	46.1340	46.5493	47.0881
2.0	48.4129	48.4512	48.5633	48.7411	48.9745
2.5	49.4488	49.4614	49.4968	49.5497	49.6484
3.0	49.8417	49.8477	49.8674	49.9034	49.9605

由表 4 可见，随着烘干进行，药材温度由初始状态逐渐升高，并在前期表现

出较明显的径向温差。随着热量不断向内部传递，各位置温度逐渐趋于稳定，表面与中心之间的温差逐步减小。

与问题一相比，变物性模型下温度场的演化不再由固定热物性参数决定，而是受到含水率持续降低的影响。随着药材失水，ρ、c_p和k均发生变化，使温度响应速度随烘干进程动态调整。

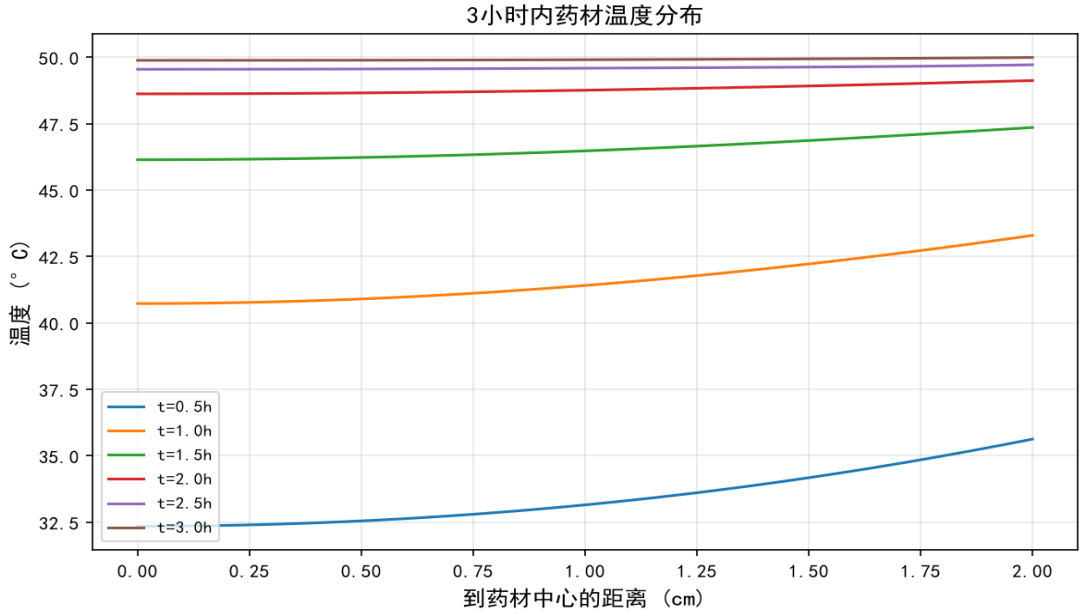


图 5 3 小时药材温度分布

图 5 给出了$3h$内药材温度随时间的变化曲线。可以看出，药材整体经历了由快速升温向缓慢变化的过渡过程。曲线逐渐趋于平缓，说明经过前期预热后，药材内部温度场逐步接近动态热平衡状态。

这一结果表明，在后续较长时间的烘干过程中，温度场逐渐由主导变量转变为影响水分迁移的辅助状态变量，而含水率场成为决定烘干终点的核心因素。

5.2.3 在 3 小时内药材水分场计算结果

相比温度场，水分迁移具有更明显的时间滞后性。表 5 给出了$3h$内不同径向位置的干基水分浓度。

表 5 3 小时内药材的水分浓度

时间/h	到药材中心的距离/cm				
	0	0.5	1	1.5	2
0.5	2.5499	2.5489	2.5261	2.3318	1.6681
1.0	2.5263	2.4962	2.3624	2.0335	1.4883
1.5	2.3899	2.3306	2.1423	1.8138	1.3635

2.0	2.1781	2.1164	1.9334	1.6384	1.2467
2.5	1.9600	1.9104	1.7470	1.4849	1.1323
3.0	1.7767	1.7274	1.5820	1.3466	1.0239

由表 5 可见，随着烘干时间增加，药材整体水分浓度持续下降，但不同径向位置的下降速度存在明显差异。表面由于直接与外部环境进行传质交换，水分优先降低；内部水分则需要经过较长距离扩散至表面，因此下降相对滞后。

这种“表面先干、内部后干”的空间特征在整个$3h$过程中持续存在，说明药材内部水分迁移具有明显空间梯度[8]。

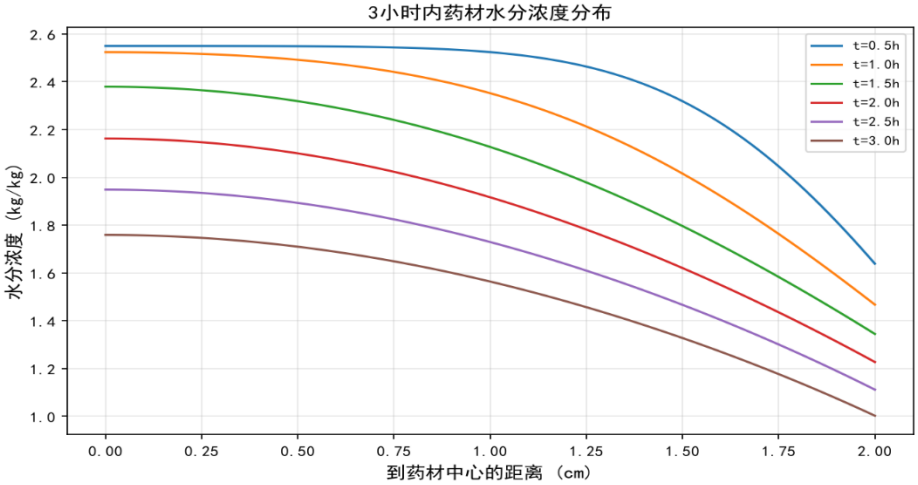


图 6 3 小时内药材水分浓度分布

图 6 进一步展示了不同位置水分浓度随时间的变化。各位置曲线总体呈下降趋势，但中心区域下降相对缓慢，表层下降更为明显，说明随着烘干进行，药材内部逐渐形成并维持水分浓度梯度。

值得注意的是，水分浓度梯度并非固定不变。随着表层不断失水，内部与表面之间的浓度差驱动内部水分持续向外迁移；与此同时，D（C，T）又随着局部状态变化而动态改变，使水分迁移速率具有明显的非线性特征。因此，采用变物

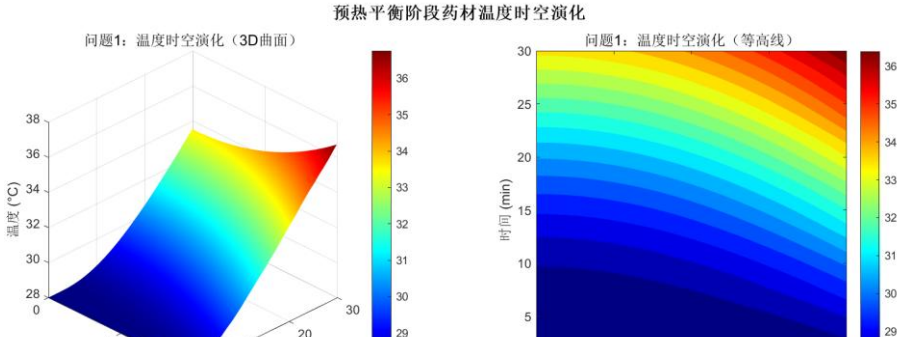


图 7 3 小时烘干过程药材温度时空演化

性模型能够比恒物性模型更加真实地描述全过程烘干。

由图 7 可见，烘干初期热量主要集中于药材表层，随后逐渐向中心传递，药材整体温度不断升高。随着时间增加，径向温差逐渐减小，温度场趋于均匀。

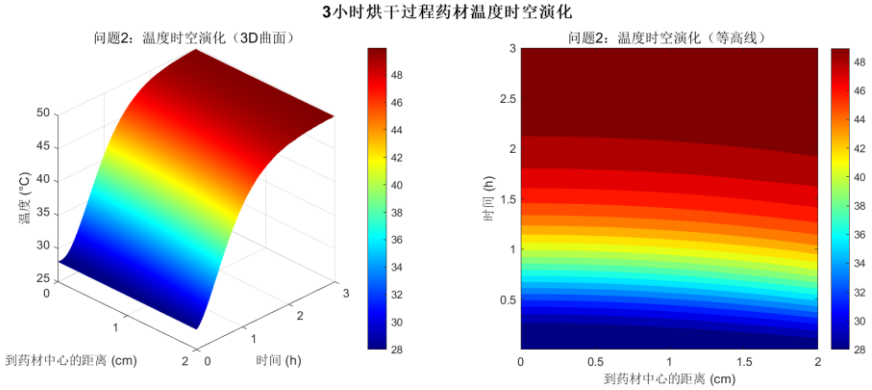


图 8 3 小时烘干过程药材温度时空演化 2

图 8 中温度变化规律与前述结果基本一致，热量由表面向内部逐步传递，中心区域升温相对滞后。随着烘干进行，整体温度升高，径向温差逐渐减小，说明计算结果具有较好的一致性。

5.2.4 在 3 小时烘干过程水分浓度时空演化

为了同时刻画水分迁移的时间和空间特征，将$0\sim3h$内各径向位置的水分浓度绘制为时空分布热力图，如图 9 所示。

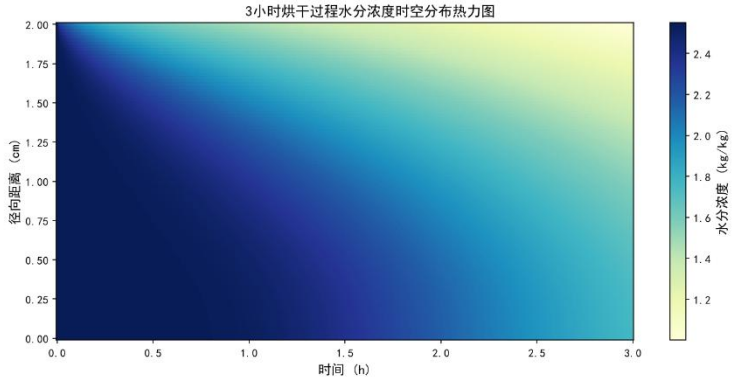


图 9 3 小时烘干过程水分浓度时空分布热力图

由图 9 可以直观看到，低水分浓度区域首先出现在药材表面，随后逐渐向内部扩展。随着时间增加，表层与内部之间的浓度差持续存在，水分迁移前沿逐步向中心推进。

热力图同时揭示了烘干过程明显的径向传质梯度：靠近表面的区域变化最早、幅度最大，而中心区域响应最慢。这说明$3h$内药材尚处于显著的内部扩散控制阶

段，不能用单一平均含水率代替真实水分场。

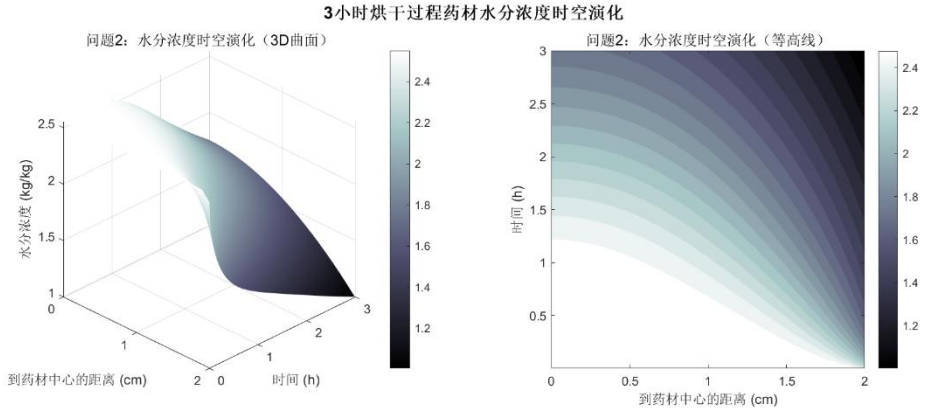


图 10 3 小时烘干过程药材水分浓度时空变化

由图 10 可见，药材表层水分浓度下降较快，并逐渐向内部扩展，形成明显的径向水分梯度。中心区域水分下降相对缓慢，表明内部水分迁移受到扩散过程限制。

此外，随着含水率降低，扩散系数D（C，T）随状态变量动态变化，使热力图中的水分迁移速度表现出非均匀性。由此可以看出，变物性并不是对问题一参数的简单替换，而是改变了整个干燥过程中的传质动力学。

5.2.5 问题二结果分析

综合表 4、表 5 及图 5～图 10 可以得到以下结论：

1. 温度场逐渐趋于稳定。烘干前期温度快速响应，随后变化趋缓，中心与表面之间的温差逐渐减小，说明药材内部热量传递较水分迁移更快。

2. 药材表层首先失水，内部水分随后向表面迁移，形成由中心向表面的径向浓度梯度。

3. 含水率变化同时改变密度、比热容、导热系数及有效扩散系数，使温度场和水分场之间形成动态耦合。因此，变物性模型能够避免恒物性假设在长时间烘干过程中的系统性偏差。

4. 随着表层水分不断降低，中心及其附近区域的水分浓度相对较高，成为后续确定烘干终点时需要重点关注的区域。

综上，问题二建立的变物性耦合模型能够同时反映药材升温、失水以及物性动态变化三个过程，并将问题一得到的预热状态自然衔接至全过程烘干，为问题三进一步确定满足全域含水率约束的最短烘干时间提供可靠的状态场基础。

5.3 问题三：满足全域含水率约束的最短烘干时间

问题二得到的结果表明，药材在烘干过程中存在明显的径向水分梯度，表层水分下降较快，而内部水分迁移相对滞后。因此，仅以药材平均含水率作为烘干终点可能导致内部局部区域仍未达到要求。

题目规定烘干结束时药材各处干基水分浓度均应低于 0.15。因此，本节以问题二得到的全过程水分场为基础，采用全域约束确定满足要求的最早时刻。

5.3.1 烘干终点判据

定义时刻t药材内部最大干基水分浓度为：

$$C_{max}(t) = \max_{0\le r\le R} C(r,t) \tag{36}$$

则烘干完成条件为：

$$t_d = inf\{t: C_{max}(t) < 0.15\} \tag{37}$$

根据要求药材内部所有位置均满足含水率要求，因此比采用平均含水率作为终点更加严格，也能够避免出现“表面已经干燥，而内部仍未达标”的情况。

在数值计算中，首先逐时刻搜索$C_{max}(t)$。当相邻两个计算时刻满足

$$C_{max}(t_n) > 0.15 \tag{38}$$

$$C_{max}(t_{n+1}) < 0.15 \tag{39}$$

时，采用线性插值确定临界时刻：

$$t_d = t_n + \frac{C_{max(t_n)-0.15}}{C_{max}(t_n) - C_{max}(t_{n+1})}(t_{n+1} - t_n) \tag{40}$$

由此避免时间步长对终点判定造成较大误差。

5.3.2 药材烘干过程的水分浓度

为分析烘干过程中不同径向位置的水分迁移规律，选取具有代表性的时间节点和径向位置提取计算结果，如表 6 所示。

表 6 药材烘干过程的水分浓度

时间/h	到药材中心的距离/cm				
	0	0.5	1	1.5	2
6	1.0290	1.0003	0.9142	0.7688	0.5499
12	0.4631	0.4502	0.4102	0.3375	0.1840
18	0.3025	0.2952	0.2724	0.2291	0.1114
…	…	…	…	…	…

烘干结束时间	0.1500	0.1477	0.1403	0.1251	0.0711

由表 6 可以看出，随着烘干时间增加，各位置水分浓度总体持续下降，但不同位置的下降速率存在明显差异。表面与外界直接发生传质交换，因此水分优先降低；内部区域则需要通过扩散不断向表面补充水分，下降速度相对较慢。

随着烘干进行，表层水分浓度已经较低，而中心区域仍保持相对较高的水分浓度。因此，决定最终烘干时间的并不是表面，而是内部最难干燥区域。

5.3.3 烘干过程水分浓度分布

为进一步展示水分场随时间的整体演化过程，绘制不同时间下药材径向水分浓度分布，如图 11 所示。

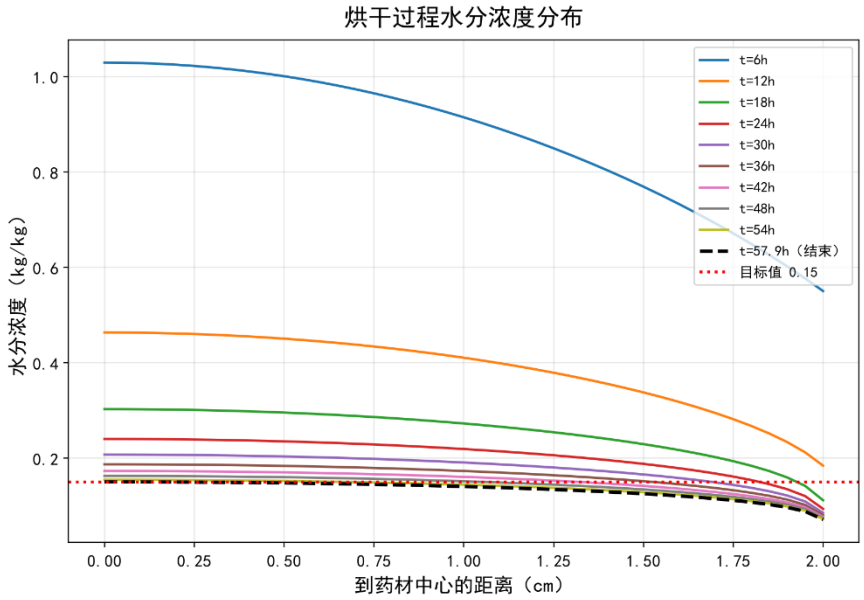


图 11 烘干过程水分浓度分布

由图 11 可见，各时刻水分浓度均表现出由中心向表面逐渐降低的总体趋势。烘干初期，药材内部水分分布差异较大；随着时间增加，水分梯度逐渐向内部推进，整体含水率持续下降。

在烘干后期，各径向位置的水分浓度逐渐接近终点要求，但中心附近仍保持相对较高的含水率。这说明随着表面水分不断减少，内部扩散逐渐成为限制整体烘干进程的关键环节。

因此，以全域最大水分浓度$C_{max}(t)$作为终点判据能够准确捕捉这一“内部滞后”现象。

5.3.4 干燥曲线及最难干燥区域判定

为进一步比较中心与表面的干燥速度，分别提取药材中心和表面的水分浓度

随时间变化曲线，如图 12 所示。

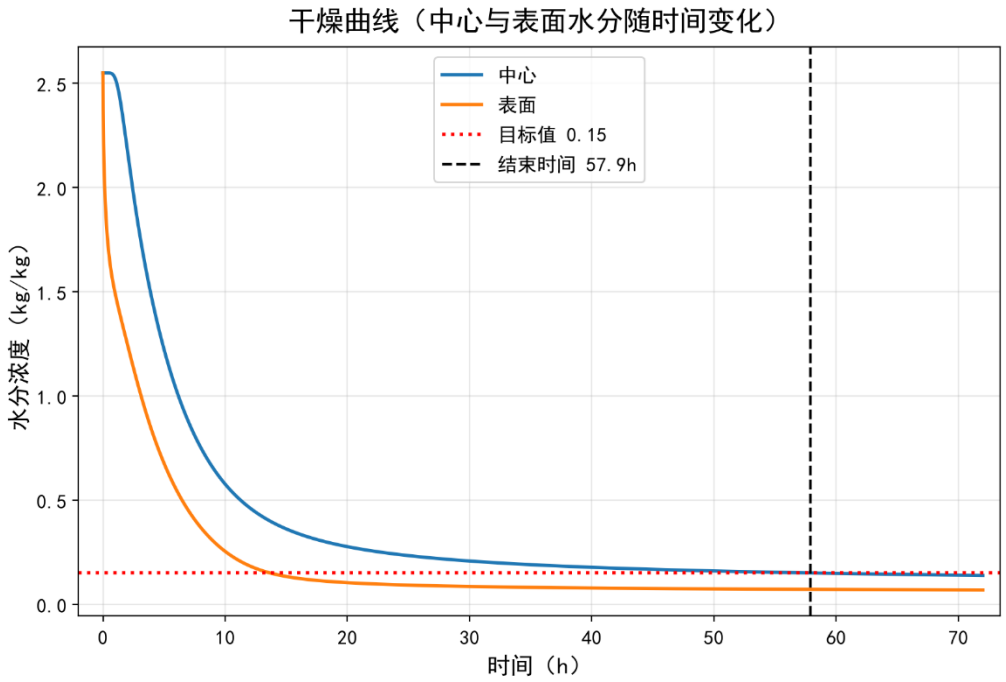


图 12 干燥曲线（中心与表面水分随时间变化）

由图 12 可以看出，中心和表面水分浓度均随时间持续下降，但两者下降过程存在明显差异。表面直接受到外部传质作用影响，水分浓度下降较快；中心则受到内部扩散距离和扩散能力的限制，水分下降明显滞后。

随着烘干时间增加，两条曲线逐渐靠近，但在达到终点之前中心仍保持较高水分浓度。由此可以确定，中心区域是该模型下的主要难干燥区域，因此采用药材内部最大水分浓度作为终点判据具有明确的物理依据。

进一步地，在终点附近对$C_{max}(t)$进行精细搜索，得到：

$$t_d = 57.8753h \tag{41}$$

即在固定尺寸、变物性条件下，药材达到“各处干基水分浓度均低于 0.15”这一要求的最短烘干时间为$57.8753h$。

5.3.5 烘干终点附近的精确判定

为验证终点判定的可靠性，进一步检查t_d前后的最大水分浓度，如表 7 所示。

表 7 烘干终点前后全域最大水分浓度验证

时刻/h	$C_{max}(t)$	是否满足$C_{max} < 0.15$
57.50	0.1504	否
57.88	0.1500	是

58.50	0.1494	是

由表 7 可验证，在t_d之前药材内部仍存在未达到要求的区域，而在t_d时首次满足全域约束。因此，

$$t_d = 57.8753h \tag{42}$$

并非由平均含水率达到阈值所得，而是由内部最难干燥区域首次达到全域含水率要求确定。

这种判定方式避免了提前结束烘干造成的内部含水率超标问题，同时避免在达到要求后继续延长烘干时间，因此在题目给定约束下具有明确的“最短时间”意义。

5.3.6 问题三结果小节

综合表 6、表 7 及图 11、图 12 可知，药材烘干过程中水分由表面向外排出，内部水分持续向表面迁移，形成明显的径向含水率梯度。随着烘干时间增加，表层首先达到低含水状态，而中心区域由于扩散距离较长，成为限制整体干燥进程的关键区域。

因此，本文不采用平均含水率，而采用

$$C_{max} < 0.15 \tag{43}$$

作为严格的全域终点判据。经终点搜索及插值计算，固定尺寸、变物性条件下的最短烘干时间为

$$57.8753h$$

该结果为问题四进一步考虑药材收缩效应提供了基准时间，同时也为评价尺寸变化对干燥效率的影响提供了统一参照。

5.4 问题四：考虑尺寸收缩的移动边界模型

5.4.1 移动边界模型建立

问题三假设药材半径保持不变，但实际干燥过程中，随着水分不断蒸发，药材会发生径向收缩，使内部水分的实际迁移距离不断减小。因此，在问题三模型基础上进一步考虑药材尺寸变化。

根据题目给出的药材半径随时间变化数据，建立$R(t)$的插值函数，并引入归一化径向坐标

$$\xi = \frac{r}{R(t)} \tag{44}$$

其中$\xi = 0$为药材中心，$\xi = 1$为实时药材表面。对于温度场和水分场，统一

记为$U(\xi,t)$，经坐标变换后，其控制方程可表示为：

$$\frac{\partial U}{\partial t}=\frac{1}{R(\mathrm{t})^{2}\xi}\frac{\partial}{\partial \xi}\left(\xi K\frac{\partial U}{\partial \xi}\right)+\xi\frac{\dot{R}}{R(t)}\frac{\partial U}{\partial \xi} \tag{45}$$

其中K分别取导热系数k和有效水分扩散系数D。式中第一项描述药材内部的传热或传质过程，第二项反映移动边界引起的空间尺度变化对内部场变量的影响。

边界条件仍采用中心对称条件和表面对流交换条件，即

$$\frac{\partial U}{\partial \xi}|_{\xi=0}=0 \tag{46}$$

在$\xi=1$处，热传递边界变为

$$-\frac{k}{R(t)}\frac{\partial U}{\partial \xi}|_{\xi=1}=h_{U}\left(U(1,t)-U_{a}(t)\right) \tag{47}$$

其中h_U根据传热、传质过程分别取相应的交换系数。数值求解采用与前文一致的有限体积法和隐式$Euler$格式，并在每个时间步根据$R(t)$更新计算区域，从而实现对药材收缩过程的动态模拟。

5.4.2 药材烘干过程的水分浓度

考虑尺寸收缩后，按照题目要求提取各时刻不同径向位置的水分浓度，结果如表 8。

表 8 药材烘干过程的水分浓度

时间/h	到药材中心的距离/cm				
	0	0.5	1	…	药材表面
6	1.0290	1.0003	0.9142	…	0.5499
12	0.4631	0.4502	0.4102	…	0.1840
18	0.3025	0.2952	0.2724	….	0.1114
…	…	…	…	..	…
烘干结束时间	0.1500	0.1414	0.1088	…	0.0643

由表 8 可见，药材水分浓度总体随干燥时间增加而降低，且表面水分下降快于内部水分，干燥过程中始终存在由中心向表面的水分浓度梯度。随着药材不断收缩，内部至表面的实际迁移距离逐渐减小，使后期水分迁移得到进一步强化，中心区域则仍是最难达到干燥要求的位置。

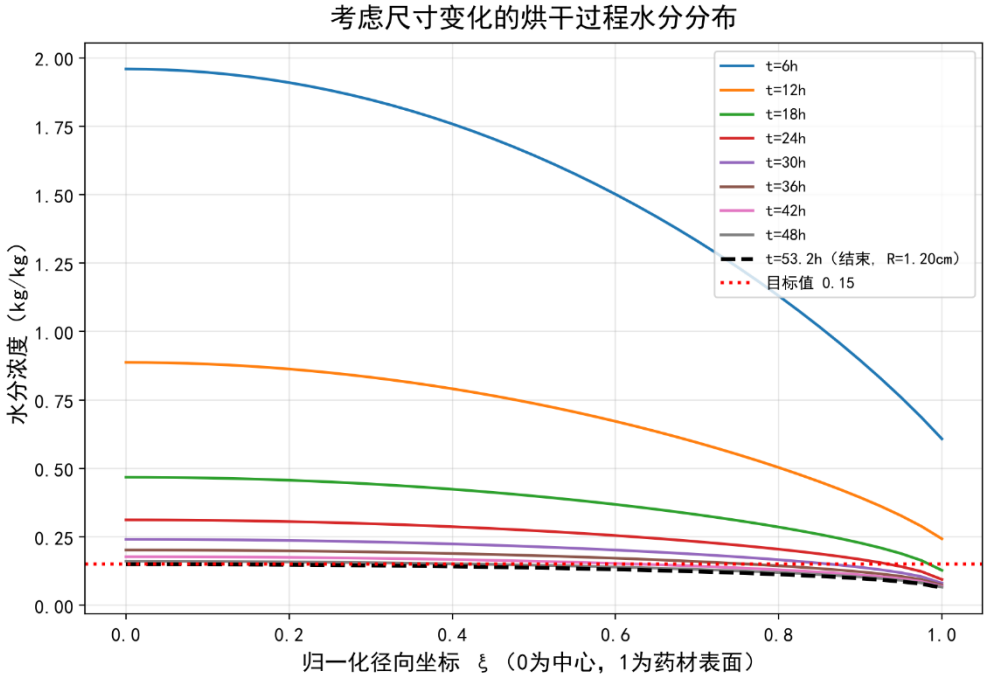


图 13 考虑尺寸变化的烘干过程水分分布

由图 13 可以直观看出，低水分区域由药材表面逐渐向内部扩展，中心区域的水分下降相对滞后。与此同时，随着$R(t)$不断减小，水分迁移区域持续收缩，后期浓度场变化速度有所提高。这说明尺寸收缩不仅改变药材几何尺寸，还通过缩短有效传质距离影响干燥动力学过程。

5.4.3 问题三与问题四干燥曲线对比

两种模型的干燥曲线总体趋势一致，均表现为表面水分浓度下降较快、中心水分浓度下降较慢。如图 14 所示。

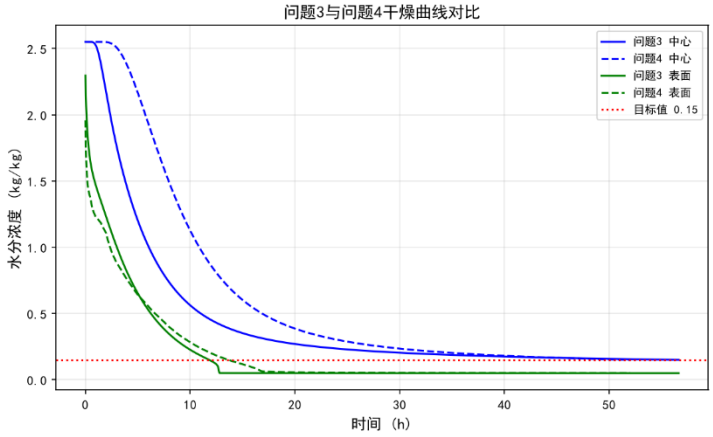


图 14 问题三与问题四干燥曲线对比

但考虑尺寸收缩后，问题四的水分浓度下降速度整体更快，且在干燥后期差异更加明显。这主要是由于药材半径不断减小，使内部水分到达表面的有效迁移距离缩短，从而加快了内部水分向表面的迁移。

以全域最大水分浓度满足

$$C_{max} < 0.15 \tag{49}$$

作为干燥终点判据，问题三与问题四分别得到

$$t_3 = 57.8753h \tag{50}$$

$$t_4 = 53.2061h \tag{51}$$

因此，考虑尺寸收缩后，烘干时间缩短

$$\Delta t = t_3 - t_4 = 4.6692h \tag{52}$$

相对缩短率为

$$\eta = \frac{t_3 - t_4}{t_3} \times 100\% \approx 8.07\% \tag{53}$$

由此可见，忽略药材干燥过程中的尺寸收缩会一定程度上高估实际烘干时间。引入移动边界后，模型能够同时描述水分迁移与药材尺寸变化的耦合效应，使干燥终点预测更加符合实际过程。

问题四最终得到的最短烘干时间为$53.2061h$，较固定尺寸模型缩短$4.6692h$，缩短幅度为8.07%。

六、 模型评价

6.1 模型优点

（1） 从传热、传质基本规律出发建立温度场与水分场耦合模型，并采用对流边界条件描述药材与热空气之间的交换过程，能够较好反映实际烘干过程。

（2） 问题二将密度、比热容、导热系数和有效扩散系数设置为温度、水分的函数，使模型能够反映烘干过程中物性变化带来的非线性影响。

（3） 问题四进一步考虑了药材干燥过程中的尺寸收缩，通过归一化坐标建立了移动边界模型，把药材半径的变化也纳入了计算范围。结果表明，考虑尺寸收缩后烘干时间由$57.8753h$降至$53.2061h$，缩短了8.07%，说明移动边界对干燥过程具有实际影响。

（4） 以全域最大水分浓度满足$C_{max} < 0.15$，作为烘干终点判据，这样能保证药材内部最难干燥的区域也达到要求，避免出现平均含水率达标但是局部含水率没有干燥的问题。

（5） 模型采用有限体积离散和隐式$Euler$时间推进，并通过$Picard$迭代处理非线性耦合，同时又结合$Thomas$算法求解离散方程，在保证计算效率的同时又能够满足较高的计算精度。

6.2 模型缺点

（1） 模型中的物性参数主要依据题目给定关系确定，实际药材还会受到品种、组织结构和初始含水率等其他影响因素，因此应用于其他药材时还需要重新标定。

（2） 本文将内部水分迁移等效为有效扩散过程，未进一步区分液态水、水蒸气及毛细作用等迁移机制，对复杂微观传质过程的描述仍不够充分。

（3） 问题四假设药材保持轴对称径向收缩，仅考虑宏观半径变化，未进一步描述孔隙率变化及局部不均匀变形。

（4） 本文主要利用题目给定数据进行模型计算与分析，若增加不同温度、初始含水率和尺寸条件下的实验数据，可进一步验证模型的适用范围。

6.3 模型推广

本文建立的模型并不局限于题目所给药材的烘干过程。对于具有近似轴对称结构的根茎类药材、果蔬及其他多孔湿物料，只需根据具体材料重新确定初始含水率、物性参数、表面传热传质系数及尺寸变化规律，即可推广使用。

在实际生产中，可进一步将环境温度、空气湿度和风速等参数作为控制变量，对不同干燥工艺进行模拟，从而确定满足含水率要求下的最优干燥时间和工艺参数。同时，可以将本文的移动边界模型进一步扩展至非均匀收缩、二维或三维结构，并结合实验数据进行参数反演和模型校正。

此外，模型还可与实际干燥设备的温湿度监测系统相结合，通过实时获取环境参数和药材状态，对干燥过程进行动态预测，实现由“固定时间干燥”向“状态驱动干燥”的转变，为中药材及农产品干燥过程的智能控制、节能优化和质量保障提供理论基础。

七、 AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于语言润色、代码纠错，详细使用情况见支撑材料。

八、 参考文献

[1] 淮秀兰,王立,倪学梓. 粒状物料干燥过程中的传热传质分析[J].北京科技大学学报,1998,(5):484-487.DOI:10.13374/j.issn1001-053x.1998.05.018.

[2] 熊耀坤,何雁,黄恺,等. 基于传热学原理研究中药材干燥数学模型及参数分析[J].世界科学技术(中医药现代化),2012,14(6):2183-2186.

[3] 张京平,刘汾阳,彭争. 干燥过程中球形果蔬边界的收缩方程[J].农业机械学报,2005,(9):71-74+78.

[4] 刘振宇,冯华,郭玉明. 方形果蔬热风干燥变形规律研究[J].农机化研究,2014,36(1):24-27.DOI:10.13427/j.cnki.njyi.2014.01.006.

[5] 薛韩玲,闫钰亭. 花椒红外-热风联合干燥动力学及传质特性[J].农机化研究,2023,45(9):209-216+225.DOI:10.13427/j.cnki.njyi.2023.09.009.

[6] 庞凌云,袁志华,詹丽娟,等. 铁棍山药片热风干燥过程中传热传质规律[J].食品与机械,2024,40(8):157-165.DOI:10.13652/j.spjx.1003.5788.2023.60171.

[7] 杨开显.黄芪切片太阳能低温干燥特性及传热传质机理研究[D].内蒙古农业大学,2023.DOI:10.27229/d.cnki.gnmnu.2023.000523.

[8] 李玉琴,沙秀秀,张喆,等. 丹参药材干燥动力学研究及其干燥过程有效成分动态变化[J].中国中药杂志,2025,50(1):128-139.DOI:10.19540/j.cnki.cjcmm.20240822.301.

附　录

一、　支撑材料的文件列表

序号	名称	说明	类型
1	图 1	问题一图注	PNG文件
2	图 2	问题一图注	PNG文件
3	图 3	问题一图注	PNG文件
4	图 4	问题一图注	PNG文件
5	图 5	问题二图注	PNG文件
6	图 6	问题二图注	PNG文件
7	图 7	问题二图注	PNG文件
8	图 8	问题二图注	PNG文件
9	图 9	问题二图注	PNG文件
10	图 10	问题二图注	PNG文件
11	图 11	问题三图注	PNG文件
12	图 12	问题三图注	PNG文件
13	图 13	问题四图注	PNG文件
14	图 14	问题四图注	PNG文件
15	solver.py	核心求解器	py文件
16	q1.py	问题一代码	py文件
17	q2.py	问题二代码	py文件
18	q3.py	问题三代码	py文件
19	q4.py	问题四代码	py文件
20	result1.xlsx	问题一结果	xlsx文件
21	result2.xlsx	问题二结果	xlsx文件
22	result3.xlsx	问题三结果	xlsx文件
23	result4.xlsx	问题四结果	xlsx文件

二、 程序代码

1. solver.py代码

```
"""统一数值求解器"""

from __future__ import annotations

import dataclasses
import pathlib
import numpy as np
import scipy.interpolate


@dataclasses.dataclass
class Config:
    r0: float = 2.0
    long: float = 25.0
    T0: float = 28.0
    C0: float = 2.55
    h: float = 25.0
    hm: float = 8.0e-7
    Nr: int = 41
    Nxi: int = 41
    picard_tolal_T: float = 1e-7
    picard_tolal_C: float = 1e-8
    picard_maxiteration: int = 20
    dt_fixed: float = 60.0
    data: str = "."
    output: str = "results"


CONFIG = Config()
```

```
def read1(path):
    import pandas as pd
    df = pd.read_excel(path)
    t = df['时间'].to_numpy(dtype=float)
    Ta = df['温度'].to_numpy(dtype=float)
    Ca = df['水分浓度'].to_numpy(dtype=float)
    index = np.argsort(t)
    return t[index], Ta[index], Ca[index]


def read2(path):
    import pandas as pd
    df = pd.read_excel(path)
    t = df['时间'].to_numpy(dtype=float)
    R = df['半径'].to_numpy(dtype=float)
    index = np.argsort(t)
    return t[index], R[index]


class Environmental_interpolation:

    def __init__(self, t, Ta, Ca):
        self.t = np.asarray(t, float)
        self.Ta = np.asarray(Ta, float)
        self.Ca = np.asarray(Ca, float)
        self._Ta    =    (scipy.interpolate.PchipInterpolator(self.t,
self.Ta, extrapolate=False) if len(self.t) >= 3 else None)
        self._Ca    =    (scipy.interpolate.PchipInterpolator(self.t,
self.Ca, extrapolate=False) if len(self.t) >= 3 else None)

    def __call__(self, t):
        scalar = np.isscalar(t)
        ttonum_arry = np.atleast_1d(np.asarray(t, float))
```

```
        ttomax = np.clip(ttonum_arry, self.t[0], self.t[-1])
        if self._Ta is not None:
            Ta = np.asarray(self._Ta(ttomax), float)
            Ca = np.asarray(self._Ca(ttomax), float)
        else:
            Ta = np.interp(ttomax, self.t, self.Ta)
            Ca = np.interp(ttomax, self.t, self.Ca)
        Ta[ttonum_arry < self.t[0]] = self.Ta[0]
        Ca[ttonum_arry < self.t[0]] = self.Ca[0]
        return (float(Ta[0]), float(Ca[0])) if scalar else (Ta, Ca)


class Radius_interpolation:

    def __init__(self, t, r):
        self.t = np.asarray(t, float)
        self.r = np.asarray(r, float)
        self._R = (scipy.interpolate.PchipInterpolator(self.t, self.r,
extrapolate=False) if len(self.t) >= 3 else None)

    def R(self, t):
        ttonum_arry = np.asarray(t, float)
        ttomax = np.clip(ttonum_arry, self.t[0], self.t[-1])
        return np.asarray(self._R(ttomax) if self._R is not None else
np.interp(ttomax, self.t, self.r))

    def Rdot(self, t, window=1800.0):
        ttonum_arry = float(t)
        a = max(self.t[0], ttonum_arry - window)
        b = min(self.t[-1], ttonum_arry + window)
        if b <= a:
            return 0.0
        return float((self.R(b) - self.R(a)) / (b - a))
```

```
# 附录参数
def para_q1(C):
    C = np.maximum(np.asarray(C, float), 1e-12)
    ρ = np.full_like(C, 820.0)
    cp = np.full_like(C, 2600.0)
    k = np.full_like(C, 0.36)
    D = 7e-9 * np.exp(-0.89 / C)
    return ρ, cp, k, D


def para_q23(C, T_c):
    C = np.maximum(np.asarray(C, float), 1e-12)
    T_k = np.asarray(T_c, float) + 273.15
    ρ = 650 + 128 * C
    cp = 1450 + 2736 * C / (C + 1)
    k = 0.21 + 0.38 * C / (C + 1)
    D = 2.4e-3 * np.exp(-0.45 / C) * np.exp(-3850 / T_k)
    return ρ, cp, k, D


def para_q4(C, T_c):
    C = np.maximum(np.asarray(C, float), 1e-12)
    T_k = np.asarray(T_c, float) + 273.15
    ρ = 760 + 90 * C
    cp = 1850 + 2150 * C / (C + 1)
    k = 0.12 + 0.20 * C / (C + 1)
    D = 4.2e-4 * np.exp(-0.30 / C) * np.exp(-3850 / T_k)
    return ρ, cp, k, D


# 线性求解
```

```
def TDMA(Subd, dia, Superd, rhs):
    Subd = np.asarray(Subd, float).copy()
    dia = np.asarray(dia, float).copy()
    Superd = np.asarray(Superd, float).copy()
    rhs = np.asarray(rhs, float).copy()
    n = len(dia)
    if n == 1:
        return rhs / dia
    for i in range(1, n):
        m = Subd[i - 1] / dia[i - 1]
        rhs[i] -= m * rhs[i - 1]
        dia[i] -= m * Superd[i - 1]
    x = np.empty(n)
    x[-1] = rhs[-1] / dia[-1]
    for i in range(n - 2, -1, -1):
        x[i] = (rhs[i] - Superd[i] * x[i + 1]) / dia[i]
    return x


def face_mean(x):
    return (x[:-1] + x[1:]) * 0.5


# 基于隐式Euler格式的固定半径扩散求解

def fixed_radius(u_past, KD, Storage, R, dt, TC_a, transfer_KD):
    u_past = np.asarray(u_past, float)
    KD = np.asarray(KD, float)
    Storage = np.asarray(Storage, float)
    N = len(u_past)
    dr = R / (N - 1)
    r = np.arange(N) * dr
    fc = face_mean(KD)
```

```
    lo = np.zeros(N - 1)
    di = np.zeros(N)
    up = np.zeros(N - 1)
    rhs = Storage / dt * u_past
    lamda0 = 4 * fc[0] / dr ** 2
    di[0] = Storage[0] / dt + lamda0
    up[0] = -lamda0
    for j in range(1, N - 1):
        rp = r[j] + 0.5 * dr
        rm = r[j] - 0.5 * dr
        αp = rp * fc[j] / (r[j] * dr ** 2)
        αm = rm * fc[j - 1] / (r[j] * dr ** 2)
        lo[j - 1] = -αm
        di[j] = Storage[j] / dt + αp + αm
        up[j] = -αp
    q = 1 / (dr / (2 * KD[-1]) + 1 / transfer_KD)
    αm = (R - 0.5 * dr) * fc[-1] / (R * dr ** 2)
    sh = Storage[-1] / (2 * dt)
    di[-1] = sh + αm + q / dr
    lo[-1] = -αm
    rhs[-1] = sh * u_past[-1] + q / dr * TC_a
    return TDMA(lo, di, up, rhs)


# 固定半径主求解

def simulatefixed_radius(t_end, ambient, problem, config=CONFIG,
dt=None, output_t=None):
    if dt is None:
        dt = config.dt_fixed
    R = config.r0 / 100
    n = config.Nr
    r = np.linspace(0, R, n)
```

```
    if output_t is None:
        output_t = np.arange(0, t_end + 0.5 * dt, dt)
    output_t = np.asarray(output_t, float)
    T = np.full(n, config.T0)
    C = np.full(n, config.C0)
    th = [0.0]
    TH = [T.copy()]
    CH = [C.copy()]
    t = 0.0
    oidx = 1
    while t < t_end - 1e-12:
        dtn = min(dt, t_end - t)
        tn = t + dtn
        Ta, Ca = ambient(tn)
        Ti, Ci = T.copy(), C.copy()
        for _ in range(config.picard_maxiteration):
            ρ, cp, k, D = para_q23(Ci, Ti)
            Tn = fixed_radius(T, k, ρ * cp, R, dtn, Ta, config.h)
            Cn = fixed_radius(C, D, np.ones(n), R, dtn, Ca, config.hm)
            eT = np.max(np.abs(Tn - Ti))
            eC = np.max(np.abs(Cn - Ci))
            Ti, Ci = Tn, Cn
            if eT < config.picard_tolal_T and eC < config.picard_tolal_C:
                break
        T, C = Ti, Ci
        t = tn
        while oidx < len(output_t) and output_t[oidx] <= t + 1e-10:
            th.append(t)
            TH.append(T.copy())
            CH.append(C.copy())
            oidx += 1
    return np.asarray(th), r, np.asarray(TH), np.asarray(CH)
```

```
def simulateQ1(t_end, ambient, config=CONFIG, dt=None, output_t=None):
    if dt is None:
        dt = config.dt_fixed
    R = config.r0 / 100
    N = config.Nr
    r = np.linspace(0, R, N)
    if output_t is None:
        output_t = np.arange(0, t_end + 0.5 * dt, dt)
    output_t = np.asarray(output_t, float)
    T = np.full(N, config.T0)
    C = np.full(N, config.C0)
    th = [0.0]
    TH = [T.copy()]
    CH = [C.copy()]
    t = 0.0
    oidx = 1
    while t < t_end - 1e-12:
        dtn = min(dt, t_end - t)
        tn = t + dtn
        Ta, Ca = ambient(tn)
        Ti, Ci = T.copy(), C.copy()
        for _ in range(config.picard_maxiteration):
            rho, cp, k, D = para_q1(Ci)
            Tn = fixed_radius(T, k, rho * cp, R, dtn, Ta, config.h)
            Cn = fixed_radius(C, D, np.ones(N), R, dtn, Ca, config.hm)
            eT = np.max(np.abs(Tn - Ti))
            eC = np.max(np.abs(Cn - Ci))
            Ti, Ci = Tn, Cn
            if eT < config.picard_tolal_T and eC <
config.picard_tolal_C:
                break
```

```
            T, C = Ti, Ci
            t = tn
            while oidx < len(output_t) and output_t[oidx] <= t + 1e-10:
                th.append(t)
                TH.append(T.copy())
                CH.append(C.copy())
                oidx += 1
    return np.asarray(th), r, np.asarray(TH), np.asarray(CH)


# 移动边界扩散求解
def moving(u_past, KD, Storage, R_m, Rdot_m_s, dt, KD_value,
transfer_KD, xi):
    u_past = np.asarray(u_past, float)
    KD = np.asarray(KD, float)
    Storage = np.asarray(Storage, float)
    N = len(u_past)
    dx = xi[1] - xi[0]
    fc = face_mean(KD)

    a = xi * Rdot_m_s / max(R_m, 1e-14)
    adv = np.zeros(N)
    adv[1:] = a[1:] * (u_past[1:] - u_past[:-1]) / dx
    rhs = Storage / dt * u_past + Storage * adv
    lo = np.zeros(N - 1)
    di = np.zeros(N)
    up = np.zeros(N - 1)
    R2 = R_m ** 2
    lamda0 = 4 * fc[0] / (R2 * dx ** 2)
    di[0] = Storage[0] / dt + lamda0
    up[0] = -lamda0

    for j in range(1, N - 1):
```

```
            xp = xi[j] + 0.5 * dx
            xm = xi[j] - 0.5 * dx
            αp = xp * fc[j] / (R2 * xi[j] * dx ** 2)
            αm = xm * fc[j - 1] / (R2 * xi[j] * dx ** 2)
            lo[j - 1] = -αm
            di[j] = Storage[j] / dt + αp + αm
            up[j] = -αp

    dr_half = R_m * dx / 2
    q = 1 / (dr_half / KD[-1] + 1 / transfer_KD)
    αm = (1 - 0.5 * dx) * fc[-1] / (R2 * dx ** 2)
    sh = Storage[-1] / (2 * dt)
    di[-1] = sh + αm + q / (R_m * dx)
    lo[-1] = -αm
    rhs[-1] = sh * u_past[-1] + Storage[-1] * adv[-1] + q / (R_m * dx)
* KD_value
    return TDMA(lo, di, up, rhs)


def simulateQ4(t_end, ambient, radius_old, config=CONFIG, dt=60.0,
output_t=None):
    n = config.Nxi
    xi = np.linspace(0, 1, n)
    if output_t is None:
        output_t = np.arange(0, t_end + 0.5 * dt, dt)
    output_t = np.asarray(output_t, float)
    T = np.full(n, config.T0)
    C = np.full(n, config.C0)
    th = [0.0]
    RH = [float(radius_old.R(0))]
    TH = [T.copy()]
    CH = [C.copy()]
    t = 0.0
```

```
    oidx = 1
    while t < t_end - 1e-12:
        dtn = min(dt, t_end - t)
        tn = t + dtn
        R_m = float(radius_old.R(tn)) / 100
        R_dot = radius_old.Rdot(tn) / 100
        Ta, Ca = ambient(tn)
        Ti, Ci = T.copy(), C.copy()
        for _ in range(config.picard_maxiteration):
            ρ, cp, k, D = para_q4(Ci, Ti)
            Tn = moving(T, k, ρ * cp, R_m, R_dot, dtn, Ta, config.h,
xi)
            Cn = moving(C, D, np.ones(n), R_m, R_dot, dtn, Ca,
config.hm, xi)
            eT = np.max(np.abs(Tn - Ti))
            eC = np.max(np.abs(Cn - Ci))
            Ti, Ci = Tn, Cn
            if eT < config.picard_tolal_T and eC <
config.picard_tolal_C:
                break
        T, C = Ti, Ci
        t = tn
        while oidx < len(output_t) and output_t[oidx] <= t + 1e-10:
            th.append(t)
            RH.append(float(radius_old.R(tn)))
            TH.append(T.copy())
            CH.append(C.copy())
            oidx += 1
    return np.asarray(th), xi, np.asarray(RH), np.asarray(TH),
np.asarray(CH)


def dtime(times, C_old, threshold=0.15):
```

```
    times = np.asarray(times, float)
    cmax = np.max(np.asarray(C_old, float), axis=1)
    for i in range(1, len(times)):
        if cmax[i] < threshold:
            if cmax[i] == cmax[i - 1]:
                return float(times[i])
            f = (cmax[i - 1] - threshold) / (cmax[i - 1] - cmax[i])
            return float(times[i - 1] + f * (times[i] - times[i - 1]))
    return np.nan


def weight_average(r, field):
    r = np.asarray(r, float)
    f = np.asarray(field, float)
    R = r[-1]
    return 2 * np.trapezoid(f * r, r) / (R * R)


def weight_average_xi(xi, field):
    xi = np.asarray(xi, float)
    f = np.asarray(field, float)
    return 2 * np.trapezoid(f * xi, xi)


def into_position(r_src, field, target):
    return np.interp(target, r_src, field)


# Excel输出

def build_df(times, positions, value, timeUnit="s"):
    import pandas as pd
    data = np.column_stack([times, value])
```

```
    cols = [f"时间/{timeUnit}"] + [f"{x:.4f}/cm" for x in positions]
    return pd.DataFrame(data, columns=cols)


# result1/result2
def result12(path, times, positions, T_values, C_values):
    import pandas as pd
    pathlib.Path(path).parent.mkdir(parents=True, exist_ok=True)
    cols = ["时间\\到药材中心的距离"] + [round(float(x), 4) for x in
positions]
    T_v = np.round(np.asarray(T_values, float), 4)
    C_v = np.round(np.asarray(C_values, float), 4)
    df_T = pd.DataFrame(np.column_stack([times, T_v]), columns=cols)
    df_C = pd.DataFrame(np.column_stack([times, C_v]), columns=cols)
    with pd.ExcelWriter(path, engine="openpyxl") as writer:
        df_T.to_excel(writer, sheet_name="温度", index=False)
        df_C.to_excel(writer, sheet_name="水分浓度", index=False)
        # 数据区域格式化为 4 位小数
        for sheet_name in ["温度", "水分浓度"]:
            ws = writer.sheets[sheet_name]
            for  row  in  ws.iter_rows(min_row=2,  max_row=ws.max_row,
min_col=2, max_col=ws.max_column):
                for cell in row:
                    cell.number_format = "0.0000"


# result3/result4
def result34(path, times, positions, values, position_labels=None):
    import pandas as pd
    pathlib.Path(path).parent.mkdir(parents=True, exist_ok=True)
    if position_labels is None:
        position_labels = [round(float(x), 4) for x in positions]
    cols = ["时间\\到药材中心的距离"] + list(position_labels)
```

```
    vals = np.round(np.asarray(values, float), 4)
    df = pd.DataFrame(np.column_stack([times, vals]), columns=cols)
    with pd.ExcelWriter(path, engine="openpyxl") as writer:
        df.to_excel(writer, sheet_name="水分浓度", index=False)
        ws = writer.sheets["水分浓度"]
        for row in ws.iter_rows(min_row=2, max_row=ws.max_row,
min_col=2, max_col=ws.max_column):
            for cell in row:
                cell.number_format = "0.0000"
```

2. q1.py代码

```
import pathlib
import numpy as np
import solver

DATA_DIR = pathlib.Path(".")
A1 = DATA_DIR / "附件 1.xlsx"
OUT = pathlib.Path("results")
OUT.mkdir(exist_ok=True)



def main():
    t_a, Ta, Ca = solver.read1(A1)
    ambient = solver.Environmental_interpolation(t_a, Ta, Ca)

    # 步长 1s/0.1cm
    times = np.arange(0.0, 1800.0 + 0.5, 1.0)
    t, r, T_old, C_old = solver.simulateQ1(1800.0, ambient,
config=solver.CONFIG, dt=1.0, output_t=times)

    positions = r * 100.0
    target = np.arange(0.0, 2.0 + 1e-9, 0.1)
    index = [int(np.argmin(abs(positions - x))) for x in target]
```

```
    solver.result12(OUT / "result1.xlsx", t, target, T_old[:, index],
C_old[:, index])
    np.savez(OUT / "Q1 data.npz", time=t, r=r, T=T_old, C=C_old)
    print("Q1 计算结果已存至：", OUT / "result1.xlsx")
    print(f"T范围: [{T_old.min():.4f}, {T_old.max():.4f}]° C")
    print(f"C范围: [{C_old.min():.6f}, {C_old.max():.6f}]")


if __name__ == "__main__":
    main()
```

3. q2.py代码

```
import pathlib
import numpy as np
import solver

DATA_DIR = pathlib.Path(".")
A1 = DATA_DIR / "附件 1.xlsx"
OUT = pathlib.Path("results")
OUT.mkdir(exist_ok=True)


def main():
    t_a, Ta, Ca = solver.read1(A1)
    ambient = solver.Environmental_interpolation(t_a, Ta, Ca)

    #共 10800 秒/3 小时，步长不变
    output_times = np.arange(0.0, 10800.0 + 0.5, 1.0)
    t, r, T_old, C_old = solver.simulatefixed_radius(10800.0, ambient,
problem=2, config=solver.CONFIG, dt=1.0,
        output_t=output_times)

    position = r * 100.0
    target = np.arange(0.0, 2.0 + 1e-9, 0.1)
```

```
    index = [int(np.argmin(abs(position - x))) for x in target]

    solver.result12(OUT / "result2.xlsx", t, target, T_old[:, index],
C_old[:, index])
    np.savez(OUT / "Q2 data.npz", time=t, r=r, T=T_old, C=C_old)
    print("Q2 结果已存至：", OUT / "result2.xlsx")
    print(f"T范围: [{T_old.min():.4f}, {T_old.max():.4f}]° C")
    print(f"C范围: [{C_old.min():.6f}, {C_old.max():.6f}]")


if __name__ == "__main__":
    main()
```

4. q3.py代码

```
import pathlib
import numpy as np
import solver

A1 = pathlib.Path("附件 1.xlsx")
OUT = pathlib.Path("results")
OUT.mkdir(exist_ok=True)


def main():
    ta, Ta, Ca = solver.read1(A1)
    ambient = solver.Environmental_interpolation(ta, Ta, Ca)

    t_end = 72.0 * 3600.0
    outputT = np.arange(0.0, t_end + 60.0, 60.0)

    t, r, T_old, C_old = solver.simulatefixed_radius(t_end, ambient,
problem=3, config=solver.CONFIG, dt=60.0,
        output_t=outputT)
```

```
    td = solver.dtime(t, C_old, threshold=0.15)
    print(f"Q3 烘干时间: {td:.2f}秒 = {td / 3600:.4f}小时")

    position = r * 100.0
    target = np.arange(0.0, 2.0 + 1e-9, 0.1)
    index = [int(np.argmin(abs(position - x))) for x in target]

    # 只保存到烘干结束时刻
    if np.isfinite(td):
        mask = t <= td + 1e-6
    else:
        mask = np.ones(len(t), dtype=bool)

    solver.result34(OUT / "result3.xlsx", t[mask], target,
C_old[mask][:, index])

    averageC = np.array([solver.weight_average(r, c) for c in C_old])
    np.savetxt(OUT / "Q3 drytime.txt", np.array([td, td / 3600]),
header="drytime(s) drytime(h)")
    np.savez(OUT / "Q3 data.npz", time=t, r=r, T=T_old, C=C_old,
avgC=averageC)
    print("Q3 结果已存至：", OUT / "result3.xlsx")


if __name__ == "__main__":
    main()
```

5. q4.py代码

```
import pathlib
import numpy as np
import solver

A1 = pathlib.Path("附件 1.xlsx")
A2 = pathlib.Path("附件 2.xlsx")
```

```
OUT = pathlib.Path("results")
OUT.mkdir(exist_ok=True)


def main():
    ta, Ta, Ca = solver.read1(A1)
    ambient = solver.Environmental_interpolation(ta, Ta, Ca)

    tr, R = solver.read2(A2)
    radius_old = solver.Radius_interpolation(tr, R)

    t_end = 72.0 * 3600.0
    outputT = np.arange(0.0, t_end + 60.0, 60.0)

    t, xi, R_old, T_old, C_old = solver.simulateQ4(t_end, ambient,
radius_old, config=solver.CONFIG, dt=60.0,
        output_t=outputT)

    td = solver.dtime(t, C_old, threshold=0.15)
    print(f"Q4 烘干时间: {td:.2f} s = {td / 3600:.4f} h")

    # 0~1.9 cm和药材表面
    target = np.arange(0.0, 1.9 + 1e-9, 0.1)
    position_labels = [round(float(x), 1) for x in target] + ["药材表
面"]

    nout = len(t)
    nposition = len(target)
    C_out = np.zeros((nout, nposition + 1))

    for i in range(nout):
        R = R_old[i]
        # 物理位置
```

```
            r_phy = xi * R
            C_row = C_old[i]

            for j, rc in enumerate(target):
                if rc <= R + 1e-9:
                    C_out[i, j] = np.interp(rc, r_phy, C_row)
                else:
                    C_out[i, j] = C_row[-1]
            C_out[i, -1] = C_row[-1]

        if np.isfinite(td):
            Mask = t <= td + 1e-6
        else:
            Mask = np.ones(len(t), dtype=bool)

        solver.result34(OUT / "result4.xlsx", t[Mask], target, C_out[Mask],
position_labels=position_labels)

        averageC = np.array([solver.weight_average_xi(xi, c) for c in
C_old])
        np.savetxt(OUT / "Q4 radius and drying.txt", np.column_stack([t,
R_old, np.max(C_old, axis=1)]),
                   header="time(s) radius(cm) Cmax")
        np.savetxt(OUT / "Q4 drytime.txt", np.array([td, td / 3600]),
header="drytime(s) drytime(h)")
        np.savez(OUT / "Q4 data.npz", time=t, xi=xi, R_cm=R_old, T=T_old,
C=C_old, avgC=averageC)
        print("Q4 结果已存至", OUT / "result4.xlsx")


if __name__ == "__main__":
    main()
```